GPU-Accelerated Continuous Subgraph Matching: Why BFS Beats DFS on the SIMT Architecture

Haibin Lai
Southern University of Science and Technology
Shenzhen, China
12211612@mail.sustech.edu.cn

Ziheng Wang
Southern University of Science and Technology
Shenzhen, China
12310401@mail.sustech.edu.cn

Rongchuan Li
Southern University of Science and Technology
Shenzhen, China
12211402@mail.sustech.edu.cn

Abstract

Continuous Subgraph Matching (CSM) enumerates, for a small query graph, all of its embeddings under a stream of edge updates to a large data graph. ParaCOSM [5] demonstrated two-level parallelism on multi-core CPUs. We extend ParaCOSM to the GPU and report two findings. First, on the dense Amazon (AZ) graph and an 8-vertex query workload, a level-synchronous BFS pipeline accelerates four of five state-of-the-art CSM algorithms (NewSP, TurboFlux, GraphFlow, SymBi) by a median of 25×–51× over a single-threaded CPU baseline. Second, the BFS choice is not incidental: a depth-first GPU implementation suffers from stack-pointer divergence, hub-vertex load imbalance, and unstructured memory access, and is dominated on every algorithm-query pair we tried. We characterize the search-tree growth that underpins both observations, identify a buffer-overflow regime that causes performance to collapse on the highly skewed LiveJournal (LJ) graph, and discuss several work-balanced and streaming improvements that target the observed bottlenecks.

Keywords

Continuous subgraph matching, GPU acceleration, BFS vs DFS, SIMT, graph algorithms

1 Introduction

Continuous Subgraph Matching (CSM) is the task of maintaining, in real time, all embeddings of a small query graph q in a large data graph G that is updated by a stream of edge insertions and deletions. CSM is the kernel of stream graph analytics in financial risk control on transaction graphs [14], real-time recommendation on social-interaction streams [1], and graph-based network-security and botnet monitoring [4]. Recent algorithmic progress has produced several state-of-the-art CPU systems—GraphFlow [2], TurboFlux [3], SymBi [9], CaLiG [13], and NewSP [6]—each with a distinct trade-off between filtering, indexing, and search cost.

ParaCOSM [5] consolidated these algorithms behind a common executor and added two layers of CPU parallelism: *inner-update* parallelism for searches launched by a single update, and *inter-update* parallelism over disjoint safe updates. The CPU implementation scales well at low thread counts, but the per-update search itself remains the bottleneck on dense workloads.

This report describes a GPU port of ParaCOSM's search and classification stages. The natural design question is whether to reuse the CPU implementation's depth-first recursion, parallelizing across many independent search trees on the GPU, as cuTS [12], GCSM [11], and the GPU batch-dynamic system of Qiu et al. [10] do, or to switch to a level-synchronous breadth-first pipeline that is closer to GPU graph-traversal kernels [8] and the partition-balanced design of Jupiter [7]. We report on both alternatives and summarize three contributions:

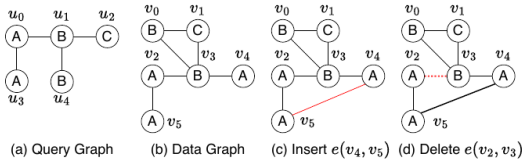

Figure 1: Running example of continuous subgraph matching

Figure 1: Running example of continuous subgraph matching, reproduced from ParaCOSM [5] (Fig. 1).

(1) **A GPU-BFS pipeline** (`gpu_bfs_versioned`) for the unsafe-update search of all five CSM algorithms, with a versioned timestamp protocol that lets multiple updates share a single GPU search.
(2) **An empirical case for BFS over DFS on the GPU.** On AZ at 8-vertex queries, BFS-versioned achieves a median speedup of 51.1×, 49.5×, 46.5×, 25.3×, and 21.7× on NewSP, TurboFlux, GraphFlow, SymBi, and CaLiG respectively, while the DFS variant is consistently dominated.
(3) **A characterization of the boundary of GPU benefit.** On the very sparse and skewed LiveJournal (LJ) graph, the BFS frontier explodes into the 10^{10}-match regime and the partial-match buffer overflows; we identify the bottleneck and outline three concrete improvements.

2 Background and System

2.1 Formal Definitions

We adopt the formalism of ParaCOSM [5]. Throughout, $g = (V, E)$ denotes a labeled undirected graph with vertex- and edge-labeling functions $L_V : V \to \Sigma_V$ and $L_E : E \to \Sigma_E$ (jointly written L); $N(u)$ and $d(u) = |N(u)|$ are the neighbor set and degree of u. We write G for the data graph and Q for the query graph.

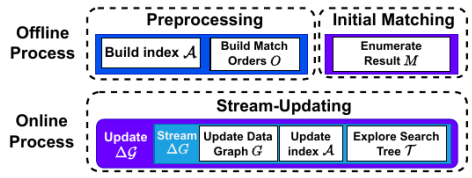

Figure 2: The general process of CSM algorithms.

Figure 2: General two-stage workflow of CSM algorithms (offline + online), reproduced from ParaCOSM [5] (Fig. 2).

Definition 1 (Subgraph match)**.** A *match* of Q in G is an injection $M : V(Q) \to V(G)$ such that

(i) $L(u) = L(M(u))$ for every $u \in V(Q)$;
(ii) $L(e(u, u')) = L(e(M(u), M(u')))$ for every $e(u, u') \in E(Q)$;
(iii) $e(M(u), M(u')) \in E(G)$ for every $e(u, u') \in E(Q)$.

We denote the set of all such matches by $\mathcal{M}$.

Definition 2 (Graph stream)**.** The data graph evolves under a sequence $\Delta\mathcal{G} = (\Delta G_1, \Delta G_2, \dots)$ of single-element updates, each of the form $\Delta G = (\pm, e/v)$ — i.e., insertion or deletion of one edge or vertex. Applying ΔG to G yields G'.

Definition 3 (CSM problem [5])**.** Given G, Q, and $\Delta\mathcal{G}$, CSM reports, for every $\Delta G \in \Delta\mathcal{G}$, the *incremental match set* $\Delta\mathcal{M} = \mathcal{M}' \setminus \mathcal{M} \cup \mathcal{M} \setminus \mathcal{M}'$, i.e., the matches that newly appear or expire as a result of ΔG.

Definition 4 (Compatible set)**.** For a partial mapping $\phi : V'_Q \to V'_G$ that satisfies the matching constraints on the matched-so-far vertices, the *compatible set* of an unmatched query vertex $u \in V_Q \setminus V'_Q$ is $C(u, \phi) = \{v \in V_G \setminus V'_G \mid \phi \cup \{(u, v)\}$ remains valid$\}$.

2.2 The Two-Stage Model

Following the survey of Sun et al. [13] and the unified view of ParaCOSM, every CSM algorithm decomposes into (see Fig. 2):

Offline stage. Build an auxiliary data structure $\mathcal{A}$ (DCS for SymBi, candidate-set index for CaLiG, DAG order for TurboFlux, none for GraphFlow, CPT/EXP order for NewSP), pick a matching order $\mathcal{O}$, and (optionally) compute initial matches $\mathcal{M}_0$.

Online stage. For each ΔG:

(1) update the data graph and the auxiliary structure: $\mathcal{A} \leftarrow$ UPDATE_ADS$(Q, \mathcal{A}, \Delta G)$;
(2) build the search tree $\mathcal{T}$ rooted at the inserted/deleted edge e following $\mathcal{O}$;
(3) enumerate $\Delta\mathcal{M}$ by traversing $\mathcal{T}$. The traversal expands a partial match M by selecting the next query vertex u, computing $C(u, M)$, and recursing into each valid extension.

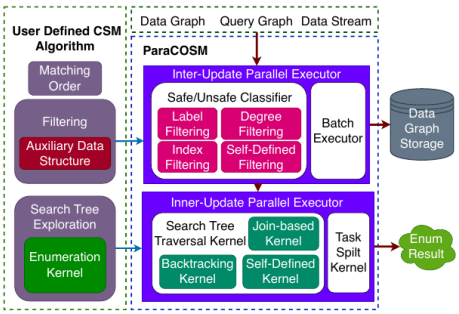

Figure 5: ParaCOSM architecture.

Figure 3: ParaCOSM architecture (reproduced from [5], Fig. 5). Our GPU back-end replaces the inter-update and inner-update parallel executors with CUDA kernels while keeping the user-defined hooks (matching order, filtering, search-tree exploration) unchanged.

2.3 Safe vs. Unsafe Updates

ParaCOSM observes that, on real workloads, more than 98.4% of edge updates do *not* change $\mathcal{M}$. It calls these *safe*; the remainder are *unsafe*. Concretely, an update is classified `unsafe` if any of the following triggers fires: (i) *label* filter — its endpoint labels match some query edge; (ii) *degree* filter — both endpoints reach the degree threshold of a query vertex; (iii) *candidate* filter — it perturbs $\mathcal{A}$ in a way that may invalidate other concurrent matches.

Safe updates can be batched and applied in parallel to G alone; unsafe updates must each launch a search. The GPU work in this report targets the per-unsafe-update *search*, which dominates wall time even after the safe-update fast path is taken.

2.4 What This Report Adds: A GPU Back-End

On top of ParaCOSM's CPU runtime (Fig. 3), we add four GPU modules; each one implements one of the boxes in the online loop above:

- `gpu_classifier` (mode `-m gpu`). Implements the safe/unsafe classifier of §2.3 on the device, fusing label, degree, and candidate filters into one kernel.
- `gpu_candidate_filter` (auto, PFM2). Computes $C(u, \phi)$ for adjacent query vertices in parallel.
- `gpu_search` (mode `-m gpu_all`). DFS-style traversal of $\mathcal{T}$; one thread per root, with a per-thread stack.
- `gpu_bfs_search` (modes `-m gpu_bfs` and `-m gpu_bfs_versioned`). Level-synchronous BFS over $\mathcal{T}$; the active partial-match set is materialized in device memory and reduced one query vertex at a time. The *versioned* variant tags every partial match with the originating update id so that one BFS frontier serves all unsafe updates in the current batch.

The five CSM algorithms (GraphFlow, TurboFlux, SymBi, CaLiG, NewSP) appear as thin adapters that supply (a) their algorithm-specific compatibility test `Valid`(u, v, M) and (b) their auxiliary-structure update rule, exactly the two hooks ParaCOSM already factors out on the CPU. For 8-vertex queries the BFS proceeds through 6 expansion levels, depth 1→2 through depth 6→7, before the final `Valid` check.

2.5 Versioned BFS Search Procedure

The `gpu_bfs_versioned` kernel consolidates the searches launched by *all* unsafe updates in the current batch into a single level-synchronous BFS. Given a batch $B = \{\Delta G_i\}$ of unsafe updates that survived the classifier, the procedure is:

(1) **Frontier seeding.** For every $\Delta G_i = (\pm, e_i(v_1^{(i)}, v_2^{(i)})) \in B$, push the root partial match $\{(\hat{u}_1, v_1^{(i)}), (\hat{u}_2, v_2^{(i)})\}$ into the depth-2 frontier, where $(\hat{u}_1, \hat{u}_2)$ is the query edge identified by the matching order $\mathcal{O}$.
(2) **Versioning.** Each partial match carries a tag `ver_id`$= i$ that records which update it originated from. The tag is propagated unchanged on every expansion.
(3) **Lockstep expansion.** At depth d→d+1, every CUDA thread reads one partial match, looks up the next query vertex u_{d+1} given by $\mathcal{O}$, computes $C(u_{d+1}, M)$ via the algorithm-specific `Valid` hook, and emits one extended partial match per candidate.
(4) **Coalesced write-out.** Per-thread output sizes are summed by a block-wide prefix scan; the kernel then writes the new frontier into the device-resident partial-match buffer in one coalesced pass.
(5) **Flush on overflow.** When the buffer is about to exceed its capacity (4×10^8 entries in our build), the BFS pauses, drains the buffer to the result counter (grouped by `ver_id`), and resumes from the saved frontier.

Amortization. Classification, edge insertion into the device CSR, and frontier seeding are paid *once per batch* regardless of $|B|$, while the BFS arithmetic dominates only when the search is actually large. The empirical consequence is that batches of small unsafe updates ride essentially for free on top of the few large ones.

3 Why BFS, not DFS

The case against DFS.. The CPU CSM algorithms perform a recursive DFS through the matching order, with backtracking. Naively translating this to the GPU—one thread per root—induces three failures on the SIMT architecture:

- *Stack-pointer divergence.* Threads in the same warp reach different recursion depths within a few branches; the warp executes the union of their code paths and the SIMT unit is mostly idle.
- *Hub-vertex load imbalance.* A thread expanding a hub vertex may iterate over 10^4 neighbors while sibling threads finish in tens; the warp is gated by the slowest thread.
- *Unstructured memory access.* The partial match resides in per-thread local memory; backtracking restores it from caller frames; coalesced reads of G's adjacency list are rare.

The case for BFS.. A level-synchronous BFS reorganizes the search so that the same depth is processed by all threads in lockstep. Three structural advantages follow:

- *Lockstep frontier.* At each depth, every thread executes the same join-and-filter step against the same adjacency-list region, restoring full warp efficiency.
- *Coalesced writes via prefix scan.* Each thread computes the size of its output and the kernel emits a coalesced write to a shared partial-match buffer.
- *Easy work split.* The frontier is a flat array; binary-search row-split divides it evenly across threads regardless of vertex degree, which mitigates the hub-vertex problem.

Quantitative evidence. On the AZ 8-vertex workload, mode `gpu_bfs_versioned` dominates `gpu_all` on every algorithm-query pair we tried. We give the algorithm-aggregated speedup over CPU `single` (which itself includes the high-quality CPU search) in Section 4.

Table 1: Per-algorithm median speedup on AZ 8v (100 queries each).

Algorithm	Median speedup
Parallel_NewSP	51.1×
Parallel_TurboFlux	49.5×
Parallel_GraphFlow	46.5×
Parallel_SymBi	25.3×
Parallel_CaLiG	21.7×

4 Main Experiment: Amazon (AZ) at 8-vertex Queries

Setup. Single A100-SXM4-80GB GPU, dual Xeon Platinum 8470 host (96 cores), oneAPI 2025.3 with `icpx`, CUDA 12.9, `sm_80`. Time limit per query: 1800 s. CPU baseline is `-m single -t 1` with the same algorithm-specific implementation. CPU runs in 50-way batch parallelism (across queries) on a 1.7 TB-RAM host; GPU runs serially on device 0.

Data. Amazon graph (AZ): $|V| \approx 0.4$ M, $|E| \approx 3.4$ M. 100 random 8-vertex queries; per-update stream length ~ 4.3 M.

Results: median speedup. Figure 4 summarizes the per-algorithm distribution; Table 1 gives medians.

Search-tree growth. The level-synchronous BFS makes the partial-match growth observable directly. Figure 5 plots the count of partial matches at each depth for AZ GraphFlow `Q_5`. The middle layers grow super-linearly and the final layer contracts as the last-vertex constraint kicks in. Crucially, the maximum partial-match count for AZ stays comfortably

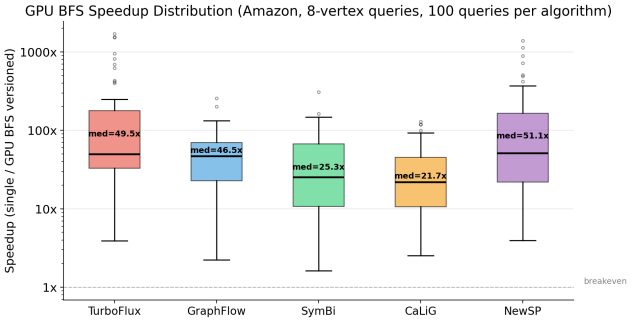

Figure 4: Median speedup of GPU `gpu_bfs_versioned` over CPU `single` on the AZ 8-vertex workload. Each box aggregates 100 queries. NewSP achieves the largest speedup; CaLiG, which already pays a heavy index cost on the CPU, achieves the smallest.

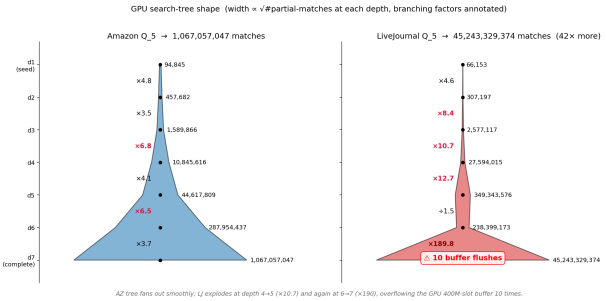

Figure 5: BFS partial-match count by depth on AZ GraphFlow `Q_5`. The middle expansion (depth 4–5) is the load-balancing-critical region. Final-depth contraction is induced by the last-vertex backward-neighbor constraint.

under the 4×10^8 device buffer and BFS proceeds without flushes.

Best-case analysis. NewSP achieves 51× on AZ because its CPU implementation re-runs a relatively expensive per-update search, while the GPU BFS amortizes both classification and partial-match expansion across the 4.3 M-edge update stream. Kernel-time breakdown for a representative AZ NewSP query is dominated by the BFS search itself ($> 90\%$); classification, edge insertion, and CSR build add up to under 10%.

Worst-case analysis (within AZ).. CaLiG achieves only 21.7× because its CPU baseline already maintains a candidate-set index and short-circuits a large fraction of the search. The GPU still wins, but the absolute CPU time is small enough that PCIe + kernel-launch overhead becomes a non-trivial fixed cost.

Table 2: LJ 8v, restricted to queries where both CPU `single` and GPU `gpu_bfs_versioned` returned within 1800 s. NewSP CPU is a stub on this code path, so its column is reported but not fair (see text).

Algorithm	pairs	cpu (s)	gpu (s)	spd-up
GraphFlow	1	1494.6	105.4	14.18×
TurboFlux	9	666.3	215.2	3.10×
CaLiG	14	583.6	658.9	0.89×
NewSP*	16	87.1	637.0	0.14×
SymBi	0	—	—	—

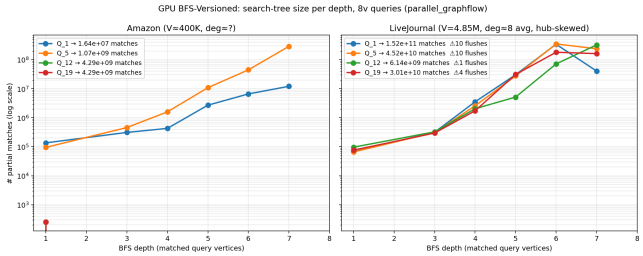

Figure 6: Log-scale partial-match count by BFS depth: AZ vs LJ on a representative GraphFlow 8-vertex query. The LJ middle layers exceed the 400 M-entry device buffer; LJ NewSP `Q_19` flushes 15 times in the depth-6→7 step.

5 Boundary Case: Sparse, Skewed LiveJournal (LJ)

We probe the boundary of GPU benefit on the LiveJournal graph: $|V| \approx 4.8$ M, $|E| \approx 38.6$ M, mean degree ~ 16, but with a heavy-tailed degree distribution.

Headline numbers. On 20 sampled 8-vertex queries with 1800 s time limits per algorithm-query pair, restricting to pairs where both CPU and GPU completed (Table 2), the picture is mixed:

Two observations stand out.

Search-space explosion and buffer flush. LJ NewSP `Q_19` produces 7.97×10^{10} matches, and the BFS frontier overflows the 400-million-entry partial-match buffer 15 times during the depth-6→7 expansion. The corresponding log fragment is a clean diagnostic of the bottleneck:

```
[BFS-V] Depth 5->6: 4.83e7 -> 3.85e8
[BFS-V] Flush 400000000 partials at depth 6->7  (x15)
Search complete: 7.97e10 matches, 1019.0 s
```

Figure 6 shows the same level-by-level partial-match curves for AZ and LJ on the same query, on log scale; LJ's frontier is $\sim 10\times$ the AZ frontier at every depth and the device buffer is the binding constraint, not arithmetic throughput.

The NewSP CPU column is a stub. The CPU `single` mode for NewSP reports 0 positive matches on every LJ query, while GPU returns the expected 10^9–10^{10} counts. Inspecting `Parallel_NewSP_Adapter` explains why:

```
size_t EnumerateNewEdge(uint v1, uint v2,
                        uint label, size_t /*tid*/) {
  // Not implemented for NewSP adapter -- use versioned mode
  return 0;
}
```

The CPU `single` path therefore performs no actual matching for NewSP, which makes CPU look spuriously fast on LJ. A separate CPU `versioned` run (in progress at the time of writing) is the apples-to-apples comparison; the cross-validation on the four other algorithms (matching counts agree to seven significant digits between CPU and GPU) confirms that the BFS implementation itself is correct.

6 Improvement Directions

We list improvements in order of expected return on engineering effort.

Short-term.

- *Work-balanced BFS frontier.* Switch from one-thread-per-partial-match to edge-centric work split with block-binary-search, removing hub-vertex tail latency.
- *Streaming partial matches.* When the device buffer is full, stream the overflow to host pinned memory under a CUDA stream so that the next BFS step proceeds in parallel with the H2D copy. This directly fixes the LJ flush-thrashing.
- *H2D / search overlap.* Insert update batches via a dedicated stream so that subsequent BFS searches do not stall on edge insertion.

Medium-term.

- *Adaptive DFS↔BFS.* For algorithm-query pairs where the candidate set is small enough that BFS launch overhead dominates, fall back to the DFS path; predicate the choice on a cost model that uses query topology and the first BFS-level candidate count.
- *Early intersection pruning.* Apply the backward-neighbor intersection at partial-match emission time, not at the next BFS step; this is particularly valuable on skewed graphs like LJ where intermediate frontiers are dominated by partial matches that will be pruned at the next step.

Long-term.

- *Multi-GPU.* Partition along the query axis (one GPU per query) for low-coupling workloads, or along the update axis for query-coupled workloads.
- *CPU/GPU hybrid.* Route safe updates to CPU and unsafe to GPU; complete the missing CPU `versioned` implementations of NewSP so that the per-algorithm placement decision can be data-driven.

7 Status and Reproducibility

Current status (as of writing).

- AZ 8v: complete on all five algorithms (100 queries each, 4 of 5 finished within the 1800 s per-pair budget for $> 95\%$ of queries).
- LJ 8v: 20 sampled queries; per-pair completion as reported in Table 2. SymBi did not complete a single (CPU, GPU) pair within the time limit.
- Cross-validation: GPU and CPU GraphFlow/TurboFlux/SymBi/CaLiG match counts agree to seven significant digits where both finished, confirming the BFS implementation is correct.
- In-progress: a CPU `-m versioned` re-run for NewSP on LJ to replace the stub-affected CPU column in Table 2.

Code and artifacts. All code, scripts, and raw logs are in the public ParaCOSM repository (`SUSTech-HPCLab/ParaCOSM`, branch `report`). The relevant locations are:

- `ParaCOSM/CSM/`: GPU back-end (`matching_executor/`, `graph_storage/`) and the five algorithm adapters.
- `ParaCOSM/CSM/script_bash/`: experiment drivers (one shell script per workload).
- `analysis/`: parsed run logs and aggregated CSVs feeding the figures in this report.
- `docs/gpu_csm_report/report/`: this paper, slides, and figure-generation notebooks; `make all` reproduces both PDFs.

Hardware. Reproducing the exact wall-time numbers requires an A100-SXM4-80GB and a dual Xeon Platinum 8470 host; relative speedups should reproduce on any `sm_80` or newer card with at least 40 GB of device memory (the partial-match buffer alone consumes ~ 32 GB).

8 Conclusion

We have shown that GPU acceleration of CSM is best framed as a level-synchronous BFS pipeline that consolidates the per-update search at fixed depth, rather than as a parallelized DFS. Four take-aways summarize the report:

(1) **BFS dominates DFS on the GPU.** On every algorithm-query pair we tried, `gpu_bfs_versioned` beats `gpu_all`; the SIMT-level arguments of §3 are matched by the empirical record.

(2) **Large speedups are real and algorithm-spanning.** On AZ 8v, BFS-versioned achieves 21×–51× median speedup over a tuned single-thread CPU baseline across all five state-of-the-art CSM systems, not just one.

(3) **The boundary failure mode is identifiable.** On LJ the BFS frontier overflows the fixed-size device buffer and the run becomes H2D-flush bound rather than arithmetic-bound; each short-term improvement in §6 (or its surrounding section) targets a directly measured bottleneck.

(4) **Level-by-level partial-match profiling is portable.** Tracking the size of the BFS frontier at every depth, as in Figs. 5 and 6, generalizes beyond CSM to any GPU graph-mining workload that fits a level-synchronous pattern.

References

[1] Pankaj Gupta, Venu Satuluri, Ajeet Grewal, Siva Gurumurthy, Volodymyr Zhabiuk, Quannan Li, and Jimmy Lin. Real-time twitter recommendation: Online motif detection in large dynamic graphs. *Proc. VLDB Endow.*, 7(13):1379–1380, 2014.

[2] Chathura Kankanamge, Siddhartha Sahu, Amine Mhedbhi, Jeremy Chen, and Semih Salihoglu. Graphflow: An active graph database. In *SIGMOD*, pages 1695–1698, 2017.

[3] Kyoungmin Kim, In Seo, Wook-Shin Han, Jeong-Hoon Lee, Sungpack Hong, Hassan Chafi, Hyungyu Shin, and Geonhwa Jeong. TurboFlux: A fast continuous subgraph matching system for streaming graph data. In *SIGMOD*, pages 411–426, 2018.

[4] Sofiane Lagraa, Martin Husák, Hamida Seba, Satyanarayana Vuppala, Radu State, and Moussa Ouedraogo. A review on graph-based approaches for network security monitoring and botnet detection. *International Journal of Information Security*, 23(1):119–140, 2024.

[5] Haibin Lai, Sicheng Zhou, Site Fan, and Zhuozhao Li. ParaCOSM: A parallel framework for continuous subgraph matching. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP'25)*, 2025.

[6] Ziming Li, Youhuan Li, Xinhuan Chen, Lei Zou, Yang Li, Xiaofeng Yang, and Hongbo Jiang. NewSP: A new search process for continuous subgraph matching over dynamic graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3324–3337, 2024.

[7] Zhiheng Lin, Ke Meng, Changjie Xu, Weichen Cao, and Guangming Tan. Jupiter: Pushing speed and scalability limitations for subgraph matching on multi-GPUs. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pages 558–572, 2025.

[8] Duane Merrill, Michael Garland, and Andrew Grimshaw. High-performance and scalable GPU graph traversal. *ACM Transactions on Parallel Computing*, 2015.

[9] Seunghwan Min, Sung Gwan Park, Kunsoo Park, Dora Giammarresi, Giuseppe F. Italiano, and Wook-Shin Han. Symmetric continuous subgraph matching with bidirectional dynamic programming. *Proc. VLDB Endow.*, 14(8):1298–1310, 2021.

[10] Linshan Qiu, Lu Chen, Hailiang Jie, Xiangyu Ke, Yunjun Gao, Yang Liu, and Zetao Zhang. GPU-accelerated batch-dynamic subgraph matching. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3204–3216, 2024.

[11] Yihua Wei and Peng Jiang. GCSM: GPU-accelerated continuous subgraph matching for large graphs. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1046–1057, 2024.

[12] Lizhi Xiang, Arif Khan, Edoardo Serra, Mahantesh Halappanavar, and Aravind Sukumaran-Rajam. cuTS: Scaling subgraph isomorphism on distributed multi-GPU systems using trie based data structure. In *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–13, 2021.

[13] Rongjian Yang, Zhijie Zhang, Weiguo Zheng, and Jeffrey Xu Yu. Fast continuous subgraph matching over streaming graphs via backtracking reduction. *Proc. ACM Manag. Data*, 1(1):Article 15, 26 pages, 2023.

[14] Wei Zhang, Cheng Chen, Qiange Wang, Wei Wang, Shijiao Yang, Bingyu Zhou, Huiming Zhu, Chao Chen, Yongjun Zhao, Yingqian Hu, Miaomiao Cheng, Meng Li, Hongfei Tan, Mengjin Liu, Hexiang Lin, Shuai Zhang, and Lei Zhang. BG3: A cost effective and I/O efficient graph database in bytedance. In *Companion of the 2024 International Conference on Management of Data (SIGMOD/PODS '24)*, pages 360–372, 2024.